

Lecture: Inference Beyond Text

CSE 585: Advanced Scalable Systems

Presenters: Kennedy McCarthy, Savini Kashmira, Preetom Kumar Biswas, Wenquan Lu

Cornfigurator: Automated Planning for Any-to-Any Multimodal Model Serving

Jeff J. Ma¹, Jae-Won Chung¹, Jisang Ahn¹, Yizhuo Liang², Runyu Lu¹, Akshay Jajoo³, Myungjin Lee³, Mosharaf Chowdhury¹

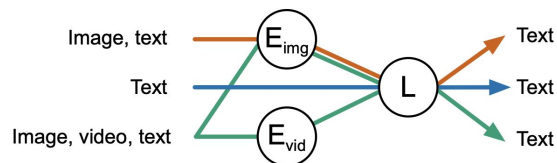
¹University of Michigan ²University of Southern California ³Cisco Research

Any-to-Any models

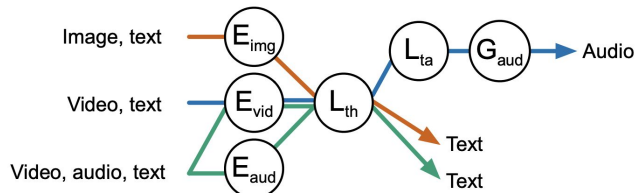
Accepts combinations of text & multimodal data -> generates text & multimodal output.

Different requests may require different components from the model. (i.e. audio encoders)

Components have different computational costs & GPU requirements



(a) Multimodal input, text output (MLLM)



(b) Multimodal input, multimodal output

Directed graph of input -> output in an Any-to-Any model

Deployment issue: Heterogeneous workloads

Table shows that throughput can differ by orders of magnitude within the same model.

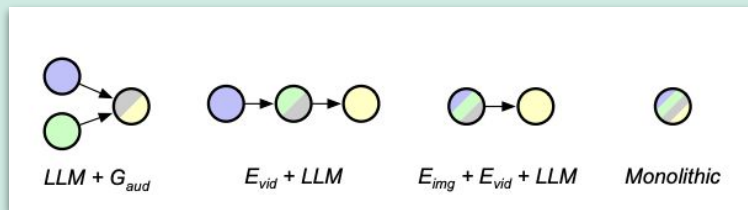
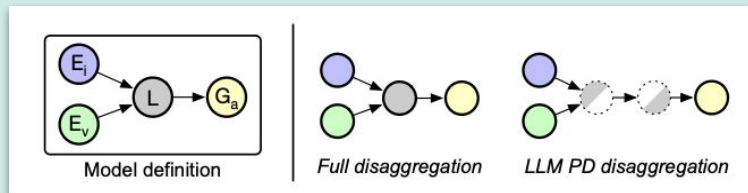
In Qwen 3 Omni, throughput of audio input is 21.42 req/sec, $\approx 180\times$ more than the throughput of audio output.

Model	Image Input	Video Input	Audio Input	Text in/out	Audio output	Image output
Qwen 3 Omni	5.43	2.93	21.43	2.15	0.12	
Qwen 2.5 Omni	15.64	1.28	34.04	1.09	0.28	
Qwen 3 VL	8.95	0.97		1.56		
Qwen 2.5 VL	12.04	0.89		1.63		
Intern VL 3	1.13	0.74		0.59		
Qwen-Image				15.67		

How should components be arranged?

Which components should be together?

- **Fully disaggregated**
 - Components can be independently scaled
- **Partially disaggregated**
 - Resources can be shared, not fully independent
- **Monolithic**
 - Components share GPU resources



Full disaggregation is not a “silver bullet”, as it still consumes resources that could be used by other components.

Other considerations

Batch size & TP

Affects throughput, latency, and GPU utilization

Communication Overhead

Splitting components across GPUs adds data-transfer costs

Large configuration space

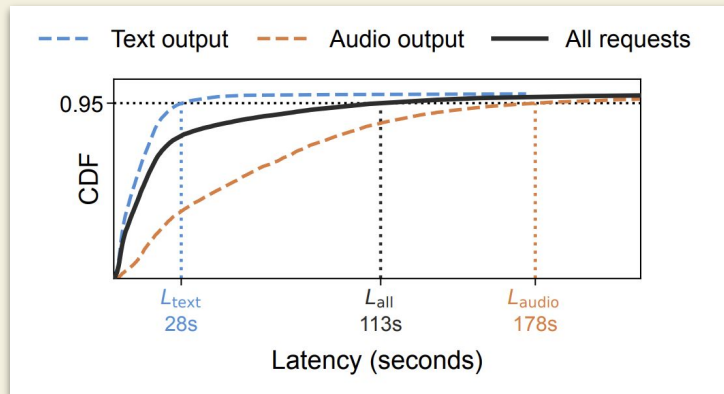
Many different configurations to filter through

Cornfigurator and Goodput

How can we find an efficient deployment configuration for a given multimodal workload?

Cornfigurator's goal is to maximize goodput

- Goodput is the throughput of requests that meet latency requirements



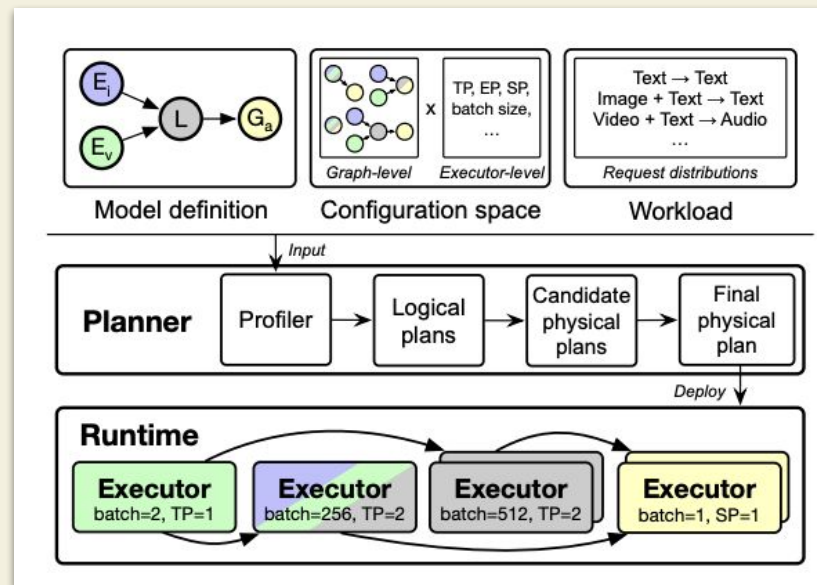
Why different request types have different latency requirements

What Cornfigurator does

Cornfigurator takes model definition, configuration space, and workload as input.

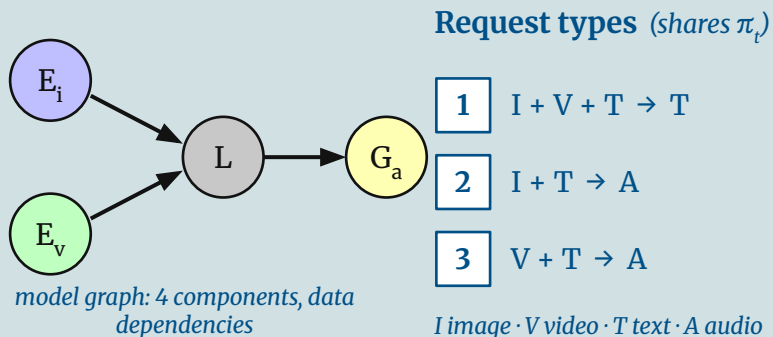
- Planner enumerates high-level, logical plans
- Initial cheap filtering to find bottleneck physical plans
- Few configurations make it to detailed simulation

Plan with highest aggregate goodput gets deployed



The planning problem

Inputs *running example, Fig. 6a*



Config space: allowed executor configs per component, profiled

GPU budget: $N = 8$

Latency target: L_t per request type

Planner

Output: one physical plan

- component grouping
- executors per group, with configs
- routing per request type
- within N GPUs

Objective: maximize goodput_t

How the planner searches

Given model graph · request mix · GPU budget · latency target per type → one physical plan, highest total goodput

① Enumerate candidate plans

Simple subplans *which components, grouped how*



Compound subplans *overlay two that share a node*



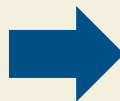
Logical plan *paths covering every request type*



Physical plan *executors, configs, routing*

each level adds detail

~10⁸ plans



② Evaluate, prune, select

Network flow *capacity ceiling*



Monte Carlo *latency → goodput per type, Pareto prune*



Simulation *queues and contention, Pareto prune*



Select *highest total goodput*

cheap first, accurate last

Simple subplans

1 Choose the request types

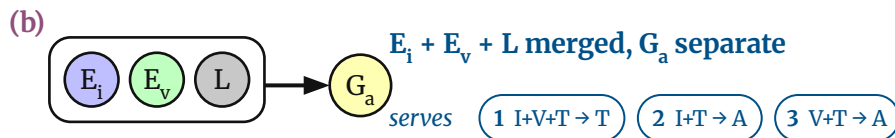
That fixes the components. Cover at least one type, use every component.

2 Keep or merge each edge

Keep = separate executors.

Merge = one executor.

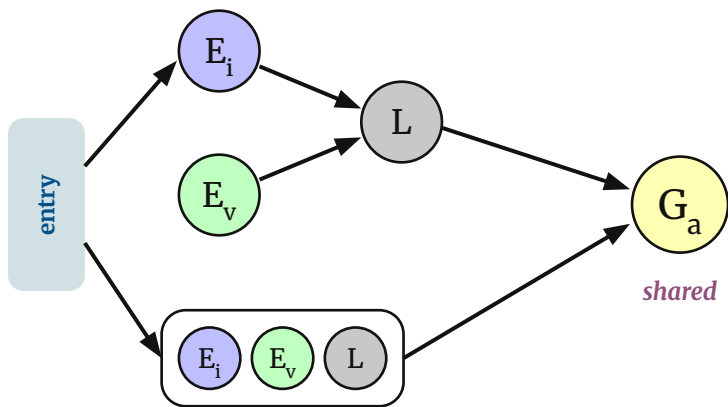
All combinations, from fully disaggregated to monolithic.



I image · V video · T text · A audio

Compound subplans

(a) + (b) overlaid, sharing G_a



routing at the entry decides which subplan a request takes

1 Overlay subplans that share a node

Up to two. Both end in G_a , so one audio pool serves both.

2 One subplan per request, end to end

No switching halfway. The split is a per-type routing probability.

3 Why it matters

Two topologies, one shared pool for the costly component. The Qwen 3 Omni plan does this.

Enumerate

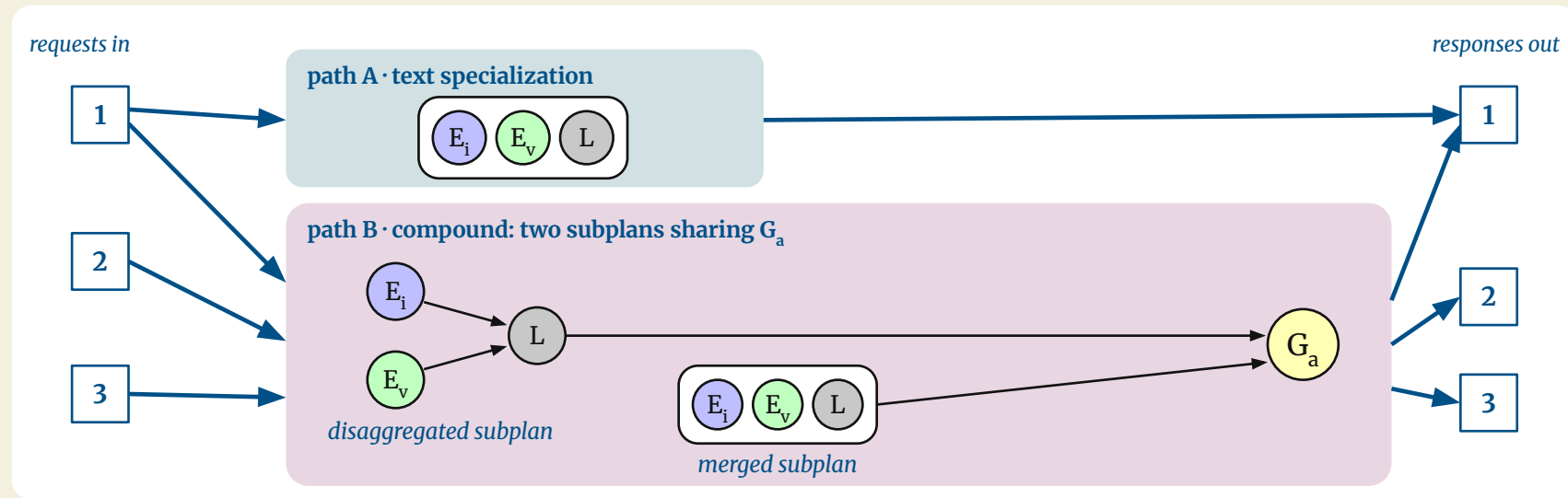
① simple subplans

② compound subplans

③ logical plan

④ physical plan

Logical plan



1 Compose parallel paths

Up to two subplans, one entry, one exit.

2 Every type needs a path

Types 2, 3 need G_a : path B only. Type 1: either.

3 Still no numbers

No GPU counts, configs or splits yet.

Enumerate

① simple subplans

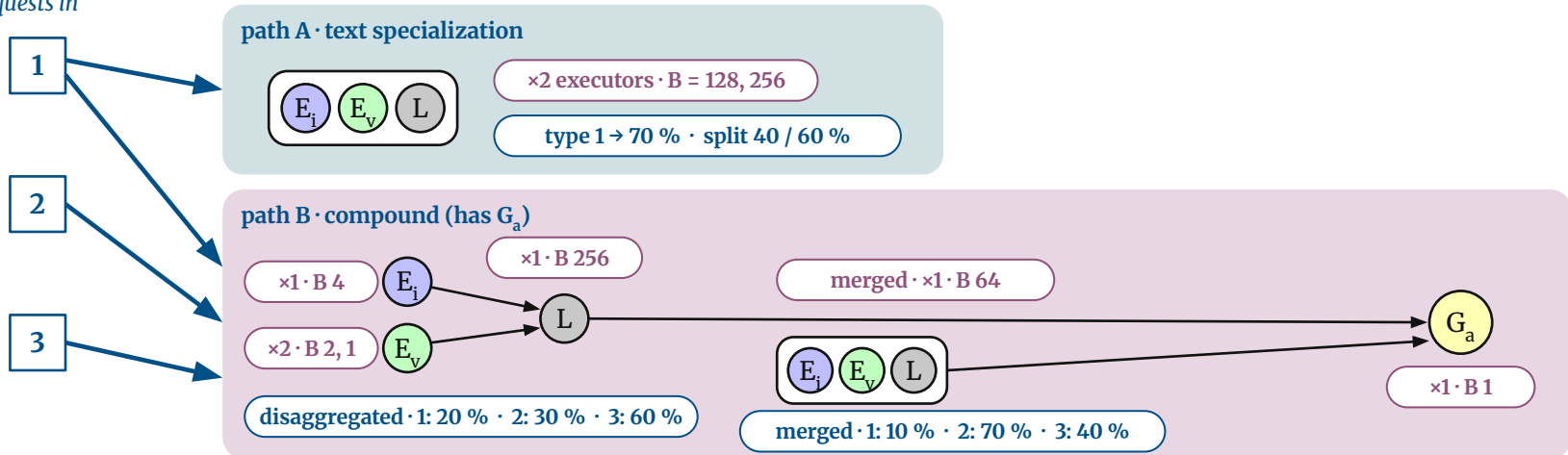
② compound subplans

③ logical plan

④ physical plan

Physical plan: executors

requests in



1 Executors per node

Count + config (B = max batch, TP). At least 1 per node, GPUs $\leq$ budget.

2 Like making change

Executors are coins worth their GPU cost.

3 Routing per request type

Split across paths and executors. 100 % per type, 10 % steps.

Enumerate

① simple subplans

② compound subplans

③ logical plan

④ physical plan

Which plan do we deploy?

after enumeration

483 M

candidate physical plans

Qwen 3 Omni on 16 GPUs

Far too many to simulate one by one.



1

Score every plan cheaply

capacity ceiling, sampled latency →
goodput per request type

483 M → 1.95 M

2

Prune hard

Pareto: drop a plan only if another beats it
on every type

→ 25

3

Simulate the finalists, pick one

queues and contention, then the highest
total goodput

→ 5 → 1

survivors: Qwen 3 Omni on 16 GPUs

Cheap first, accurate last. Every stage prunes before the next.

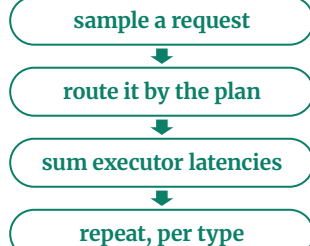
Evaluating a plan

1 Capacity *network flow*



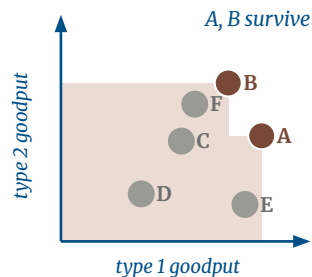
Raise the request rate until a node saturates: that is the ceiling. Here G_a fills first.

2 Latency *Monte Carlo*



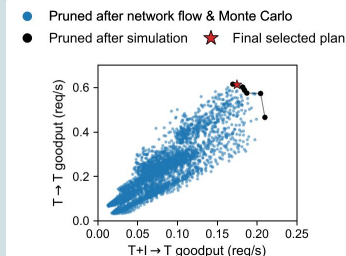
Sample requests, route them, add up executor latencies. Per type: goodput = rate $\times$ share on time.

3 Pareto prune *goodput vectors*



Drop a plan only if another beats it on every type. Millions $\rightarrow$ tens.

4 Simulate & select *finalists only*



Queues and contention, finalists only. Pareto again, then the highest total goodput.

Adapting to change

What changes		What Cornfigurator does	Re-profiling
GPU budget changes <i>e.g. 8 → 16 GPUs</i>	➡	Re-run the planner with the new budget. The profiles stay valid.	none <i>seconds</i>
Request mix shifts <i>e.g. image share 25 % → 50 %</i>	➡	Re-weight the recorded per-request samples and replan. No new measurements.	none <i>seconds</i>
New request type, or very different request sizes	➡	Profile that type only, then replan.	that type
New model or new GPU type	➡	Profile everything from scratch, then replan. Rare.	everything <i>node-hours</i>

Replanning takes seconds. Only re-profiling costs node-hours, and most changes need none.

Implementation



Planner & runtime

≈5K lines of Rust

Runtime-agnostic. Runs on Cornserve.



Profiler

Steady-state window only

Saturating load per config.
Queueing subtracted via traces.



Latency model

≈10 ms median per edge

Corrects for occupancy,
CPU/GPU overlap, and
transfers.

how one profile is measured

sweep configs

saturating rate

steady-state window

subtract queueing

Setup and goodput results

Model	Workload	Goodput gain	Plan insight
Qwen 3 Omni 30B <i>multimodal to text / audio</i>	ServeGen	1.24×–2.73×	Specialized paths, shared audio
InternVL 3 38B <i>multimodal to text</i>	ServeGen	1.12×–6.06×	Separate the large encoder
Qwen 3 VL 32B <i>multimodal to text</i>	ServeGen	1.00×–6.32×	Colocate the small encoder
Qwen-Image <i>text to image</i>	DiffusionDB	Monolithic chosen	Light prefill, heavy diffusion

8 or 16 × A100-80GB

GPUs, up to two nodes

Cornserve

same runtime for every plan

1.00×–6.32×

goodput vs. baselines

Qwen 3 Omni plan

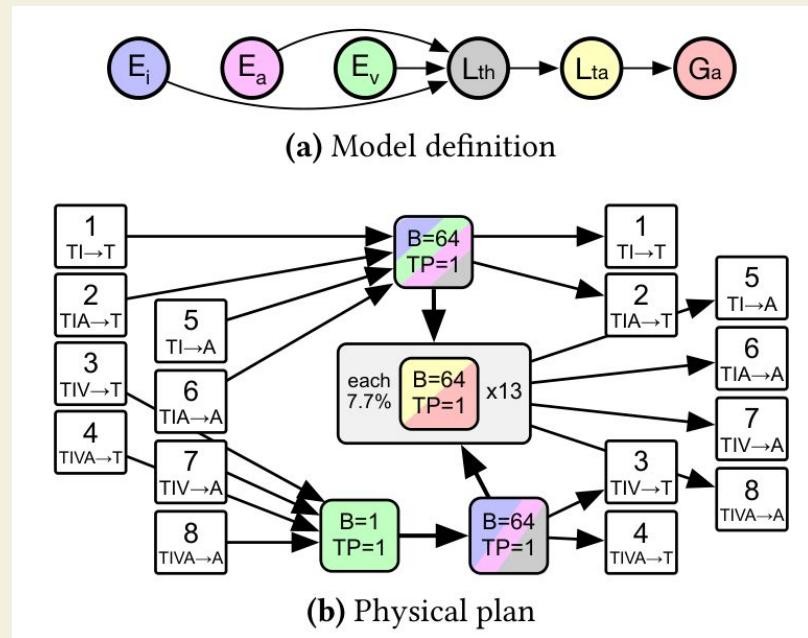


Fig. 10: model definition and the selected plan

16 GPUs
budget

2/3
audio-output requests

1 Video specialization

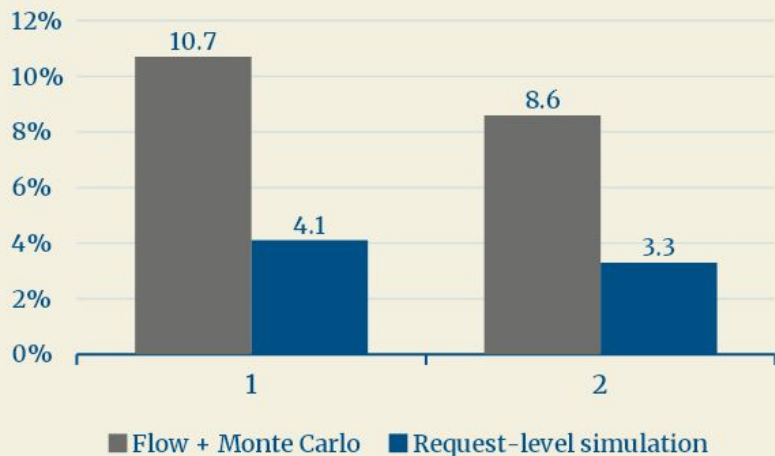
A dedicated encoder and path for heavy video requests.

2 Shared audio capacity

13 talker + vocoder replicas serve both paths.

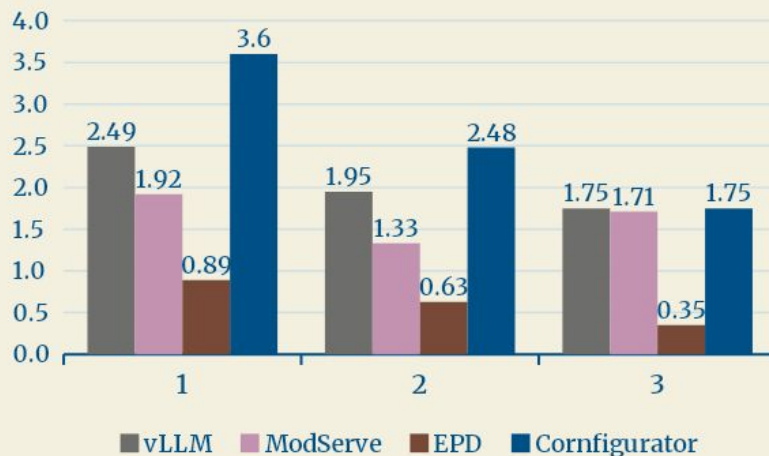
Accuracy & workload drift

Goodput prediction error, Qwen 3 Omni (Table 3)



Cheap flow + Monte Carlo estimates screen the search; request-level simulation cuts mean error from 10.7% to 4.1% for the finalists.

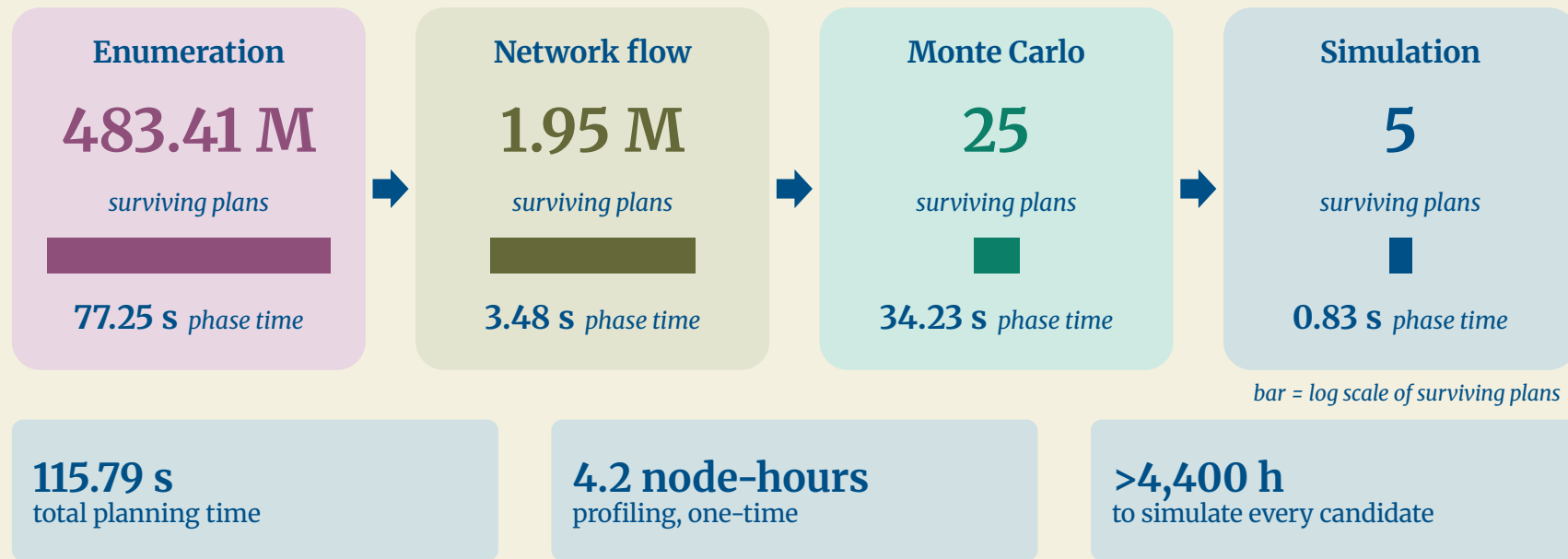
Workload drift: InternVL 3, 16 GPUs (Table 4)



Goodput in req/s. Replanning per image-input mix reuses the profiles and takes seconds; it matches or beats every baseline and ties the vLLM plan at 75%.

Planning overhead

Qwen 3 Omni, 16 GPUs — the hardest planning case (Table 6)



Takeaways & discussion

One search: topology, executor settings, allocation, and routing.

1 Deployment is workload-dependent

Colocate, separate, or specialize — model and workload decide.

2 Simulation only for the finalists

Cheap estimates prune the search first.

3 Objective: on-time completions

Gains depend on the model and the baseline.

Practical limits

- Profiling cost, workload representativeness
- Search growth on larger graphs

Questions for production

- When to replan and redeploy?
- A minimum goodput per request type?

LegoDiffusion: Micro-Serving Text-to-Image Diffusion Workflows

Lingyun Yang^{†*}, Suyi Li^{†*}, Tianyu Feng[†], Xiaoxiao Jiang[†], Zhipeng Di, Weiyi Lu, Kan Liu, Yinghao Yu, Tao Lan, Guodong Yang, Lin Qu, Liping Zhang, Wei Wang[†]

[†]Hong Kong University of Science & Technology
Alibaba Group

Real Codename: DiFlow



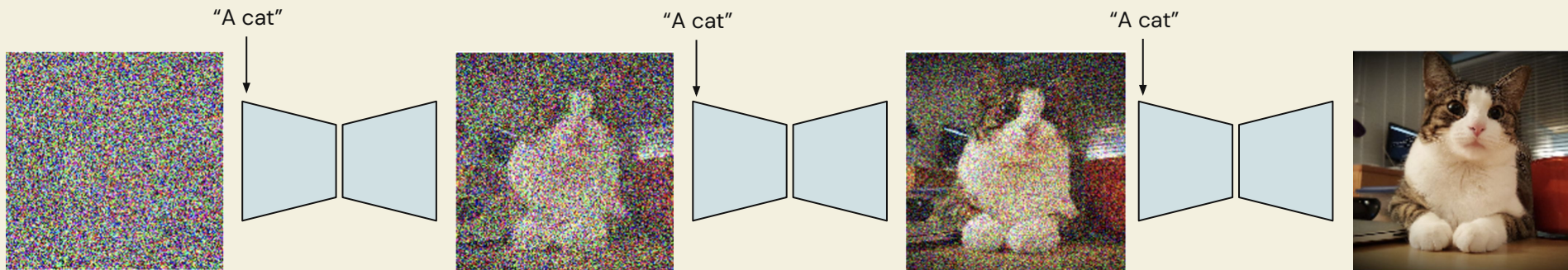
Promise

To improve Modern Text-to-Image (T2I) generation workflow

Scaling

Model Sharing

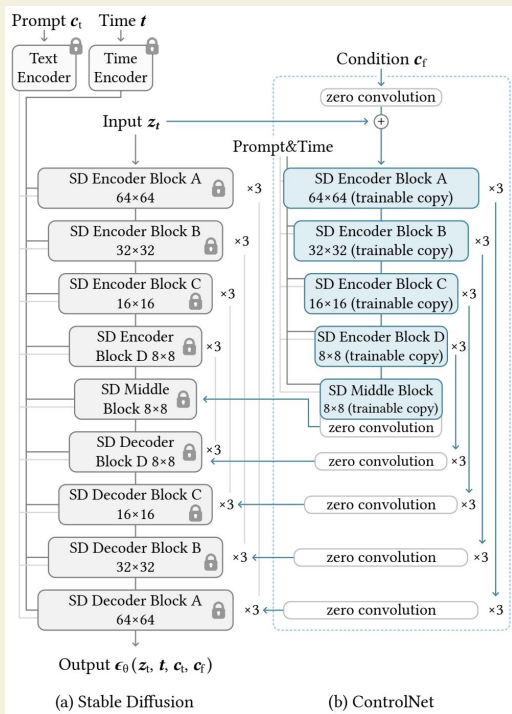
Parallelization



Iterative Denoising Steps

Background – ControlNet and Adapters

ControlNet

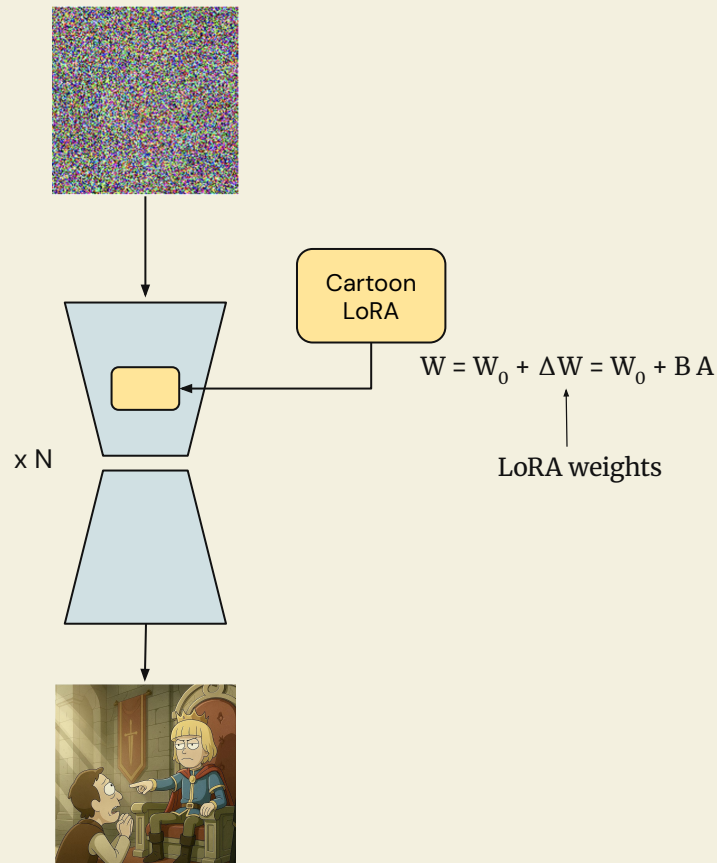
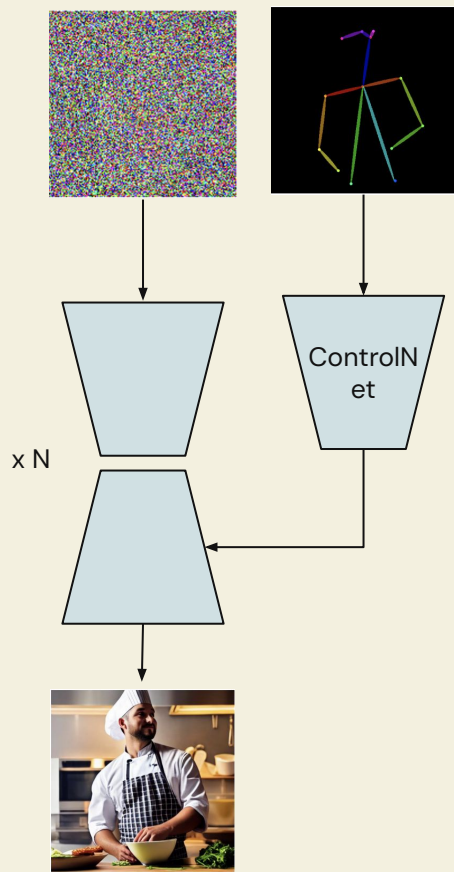


LoRA Adapters

$$W = W_0 + \Delta W = W_0 + B A$$

LoRA weights

Background – ControlNet and Adapters

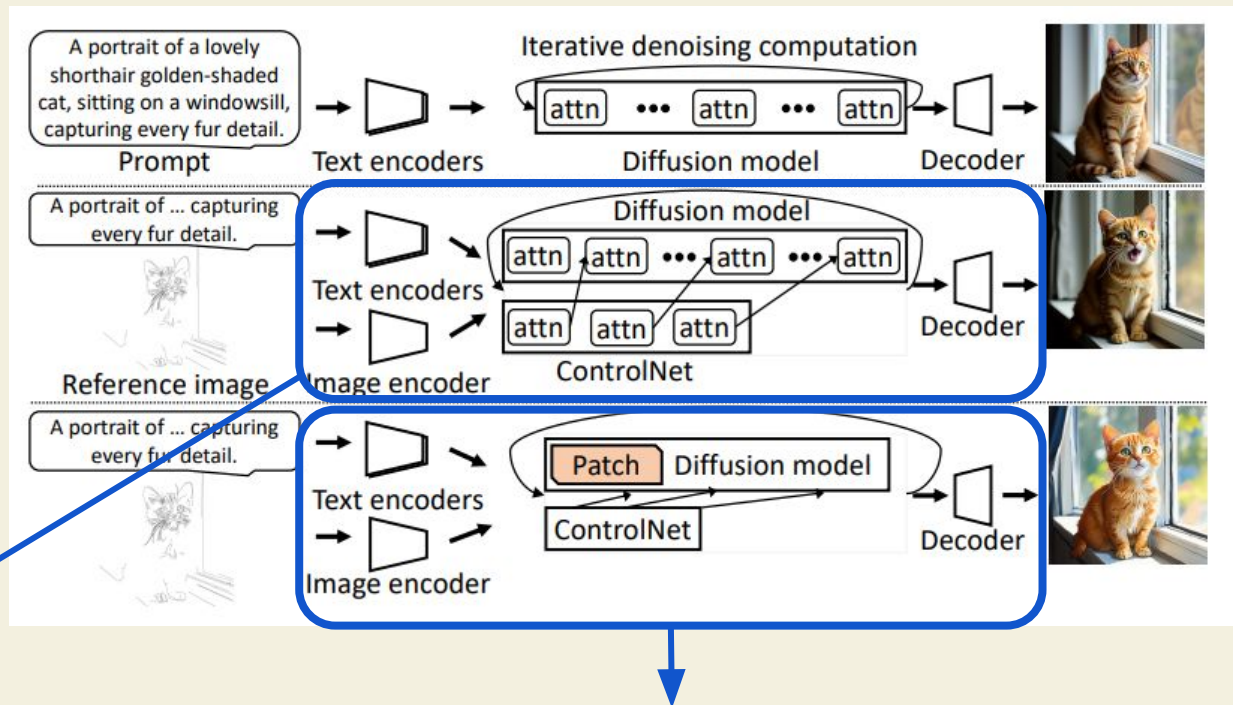


Background – ControlNet and Adapters

Basic Diffusion workflow

Adapters for refinement

(visual attributes, aesthetics, spatial structure)



Parallel Execution Adaptors (ControlNet)

Weight-Patching Adaptors (LoRA)

Workflows with adapters

performance-oriented parallelization strategies

(Intra-Node) Latent Parallelism (CFG)

(Inter-Node) ControlNet Parallelism

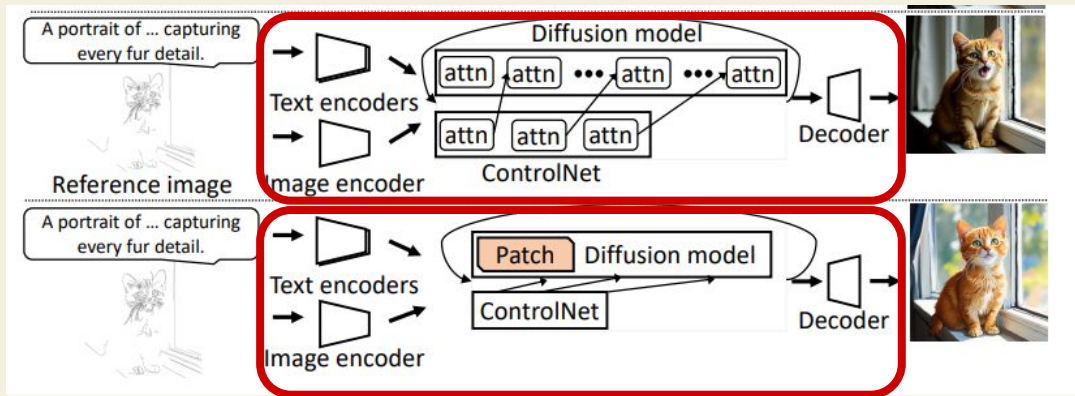
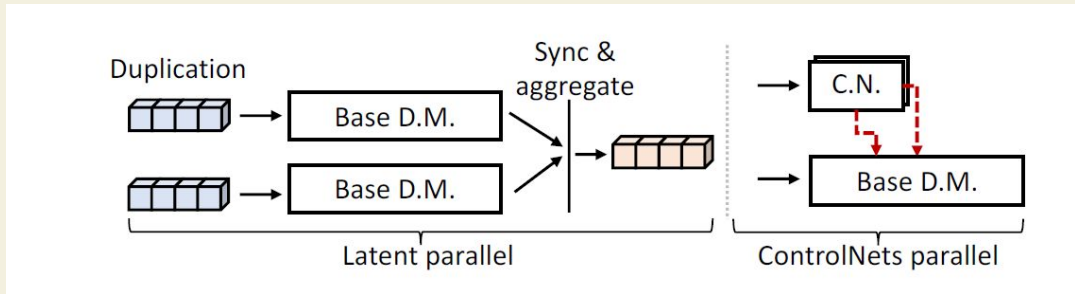
Asynchronous LoRA loading



Synchronization struggles

Inefficient Scheduling

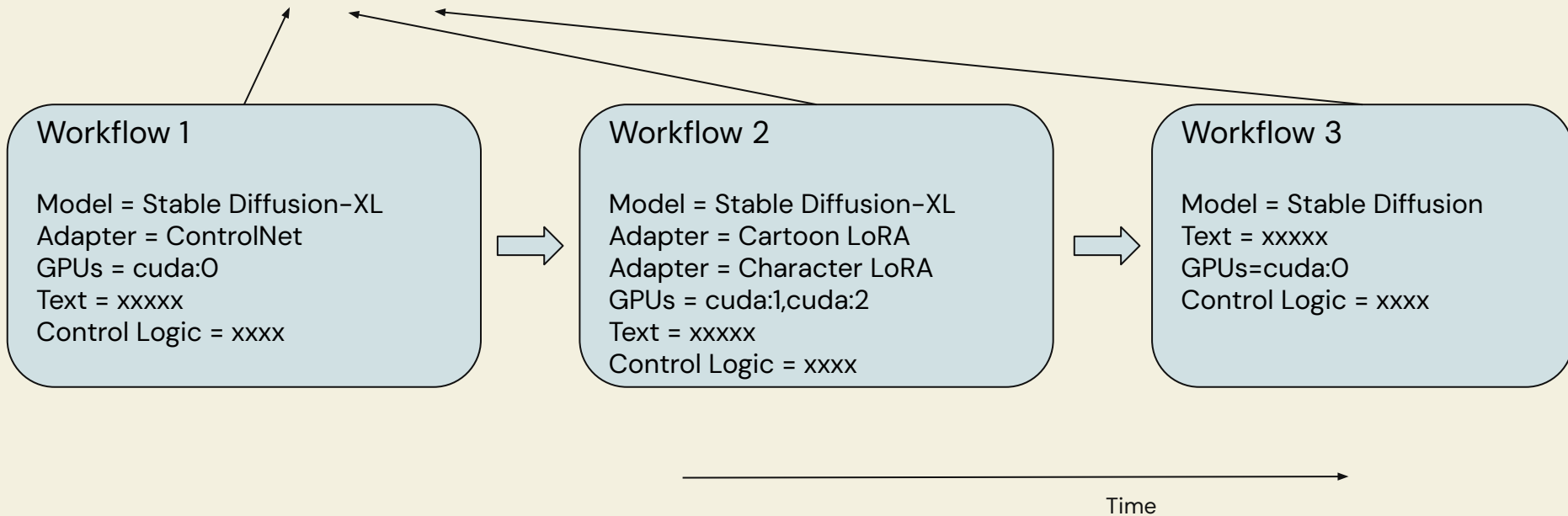
Non-deterministic Timing,



Monolithic Serving

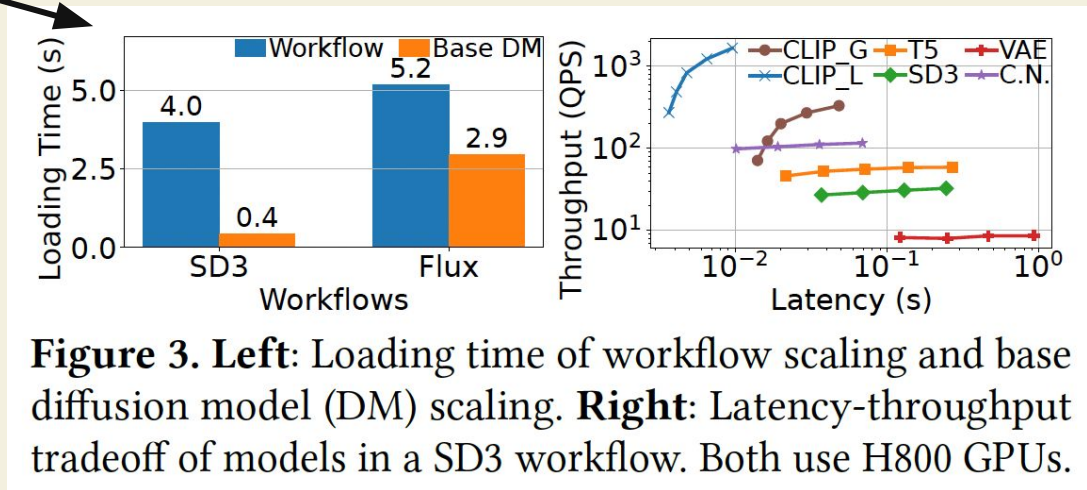
Monolithic Serving

Encapsulate the entire generation pipeline, comprising the base model, adapters, and control logic, into a **monolithic execution unit**.



Inefficiency of Monolithic Serving

- Inefficient Scaling via Full Replication
- Runtime Inefficiency
- Inability to Share Common Models
- System Fragility and Maintenance Overhead



forfeits the individual tradeoffs

Inefficiency of Monolithic Serving

- Inefficient Scaling via Full Replication
- Runtime Inefficiency
- Inability to Share Common Models
- System Fragility and Maintenance Overhead

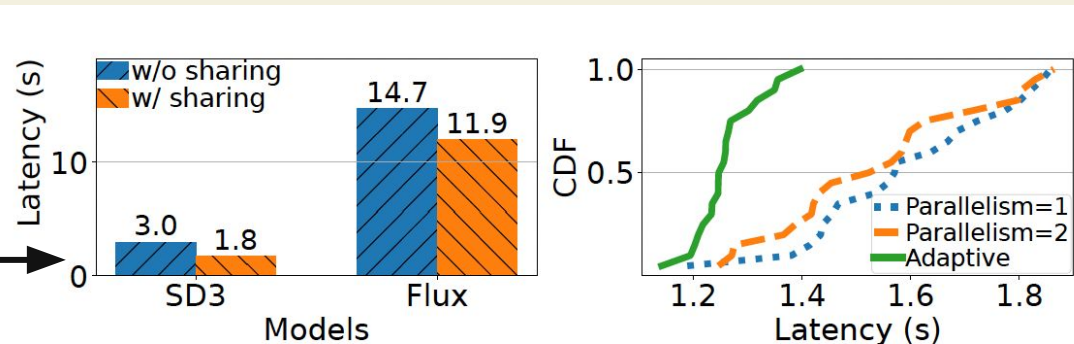
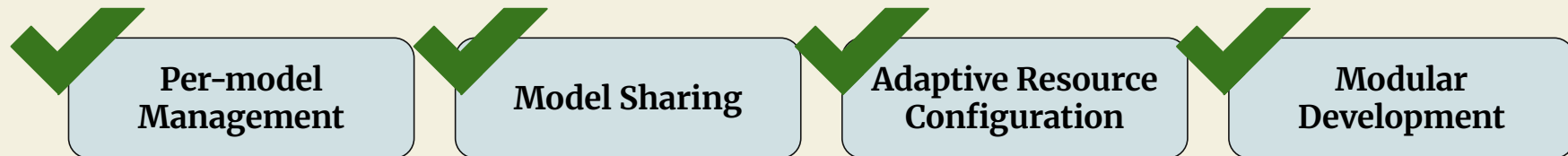


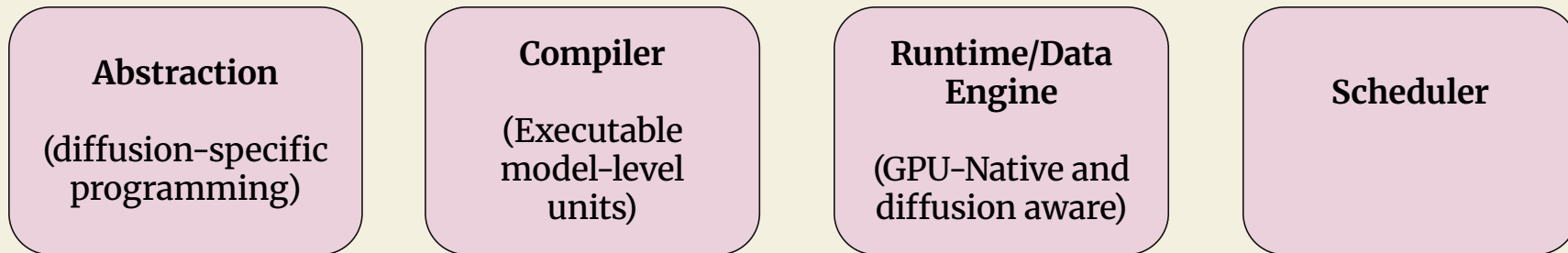
Figure 4. Left: Model sharing reduces request latency. **Right:** Adaptive parallelization reduces request latency.

Micro-Serving



But, micro-serving *diffusion* workflows cannot be built by directly reusing **existing microservice systems**.

LEGO Diffusion



LEGO Diffusion (DiFlow)

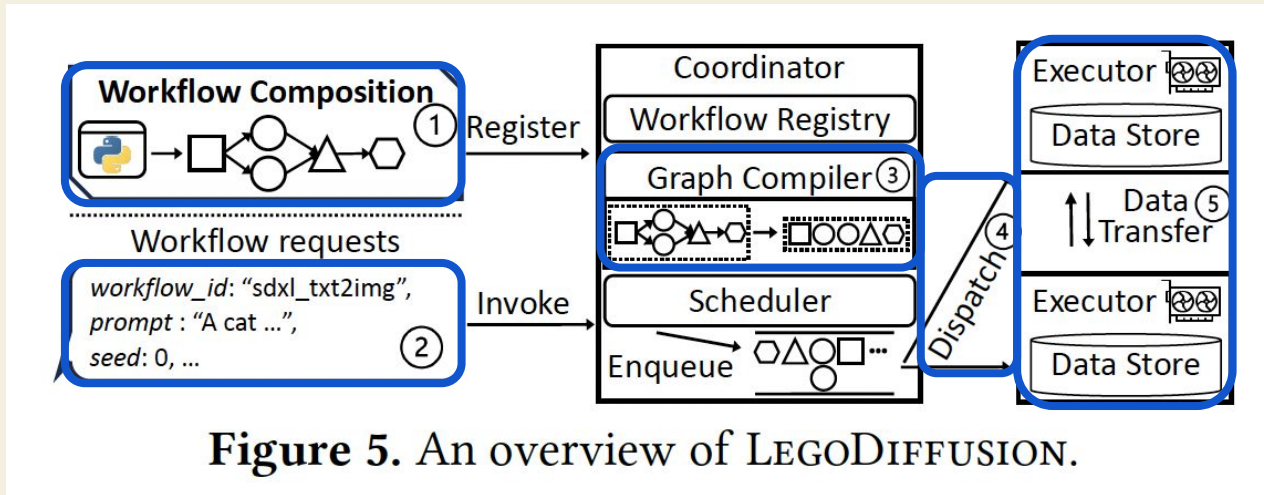


Figure 5. An overview of LEGO DIFFUSION.

- At the frontend, model developers integrate individual models, and workflow developers compose these models into workflows and register them with the system.
- End users invoke registered workflows by submitting requests with inputs.
- At the backend, the graph compiler transforms a workflow into a set of loosely coupled workflow nodes.
- The workflow nodes are dispatched by the scheduler across a cluster of distributed executors..
- Each executor owns one GPU and uses an efficient data engine for inter-node communication.

LEGO Diffusion – Programming Model



Model Developers

Implement model specific logic without reasoning about how their models are composed into a workflow.



Workflow Developers

Assemble models into end-to-end workflows by instantiating models and invoking them; **they do not manually wire a DAG.**

Implicit workflow programming model:
graph compiler infer the workflow based on
model invocation and I/O interfaces declared.

Model developer only implements three methods

```
25 ## Model developers start here ##
26 class Flux(Model):
27     def setup_io(self):
28         # define inputs
29         self.add_input("latents", torch.Tensor)
30         self.add_input("prompt_embeds", torch.Tensor)
31         # define "deferred" inputs, detailed in Sec. 4.3.2
32         self.add_input("controlnet_inputs", torch.Tensor,
33                         deferred=True)
34         # define outputs
35         self.add_output("noise_pred", torch.Tensor)
36
37     def load(self, model_path, device):
38         model = SD3Transformer2DModel.from_pretrained(
39             ...
40         ).to(device)
41         return {"transformer": model}
42
43     @torch.no_grad()
44     def execute(self, model_components, **kwargs):
45         transformer = model_components["transformer"]
46         noise_pred = transformer(**kwargs)[0]
47         return {"noise_pred": noise_pred}
```

setup_io(): Declares the model's typed inputs and outputs, which are visible to the compiler

load(): Initialize the model on a given device.

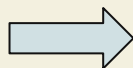
execute(): Runs inference

Workflow to DAG

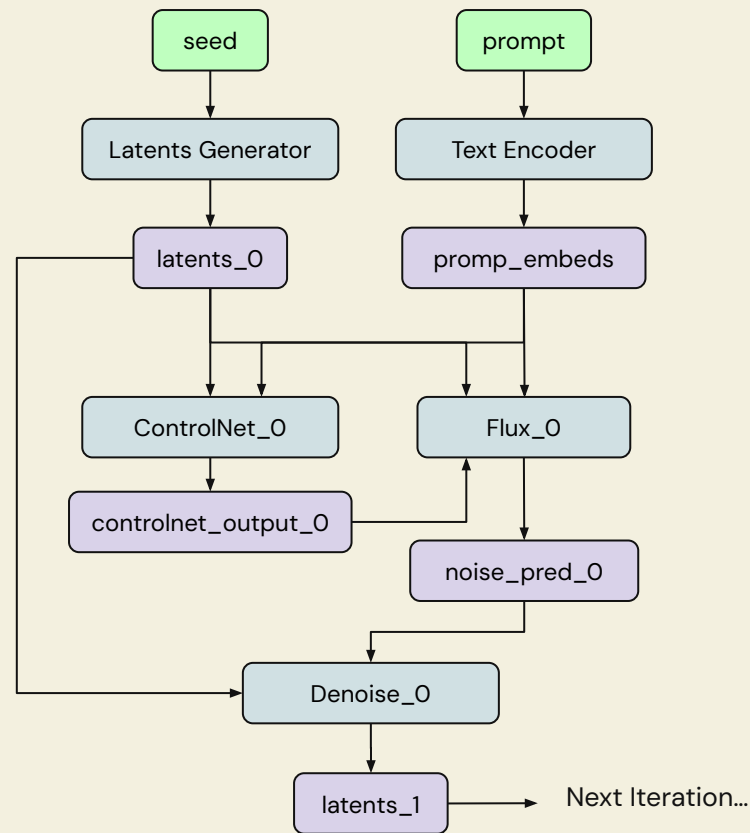
```
1 # create a workflow instance
2 workflow = Workflow(name="flux_txt2img_workflow")
3 # initialize models. All inherit the Model class
4 latents_generator = LatentsGenerator()
5 text_enc = FluxTextEncoder(model_path=model_path)
6 flux = Flux(model_path=model_path)
7 controlnet = ControlNet(model_path=controlnet_path)
8 lora = LoRA(model_path=lora_path)
9 vae = FluxVAE(model_path=model_path)
10 # initialize input placeholders for the workflow
11 seed = workflow.add_input(name="seed", data_type=int)
12 prompt = workflow.add_input(name="prompt", data_type=str)
13 num_denoising_steps = workflow.add_input(name="
    num_denoising_steps", data_type=int)
14 ref_image = workflow.add_input(name="ref_image",
    data_type=Image)
15 # add_patch() registers a LoRA adapter with the flux,
16 flux.add_patch(lora)
17 # Invoke models and establish their I/O dependencies.
18 # Model invocation is expressed via __call__().
19 latents = latents_generator(seed)
20 prompt_embeddings = text_enc(prompt)
21 ref_image = vae(image=ref_image, mode="encode")
22 # Perform iterative denoising computation.
23 for i in range(num_denoising_steps):
24     controlnet_outputs = controlnet(latents,
        prompt_embeddings, ...)
25     noise_pred = flux(latents, prompt_embeddings,
        controlnet_outputs, ...)
26     latents = denoise(noise_pred, latents)
27 output_img = vae(latents, mode="decode")
28 workflow.add_output(output_img, name="output_img")
```

Figure 7. A simplified diffusion workflow using Flux [34].

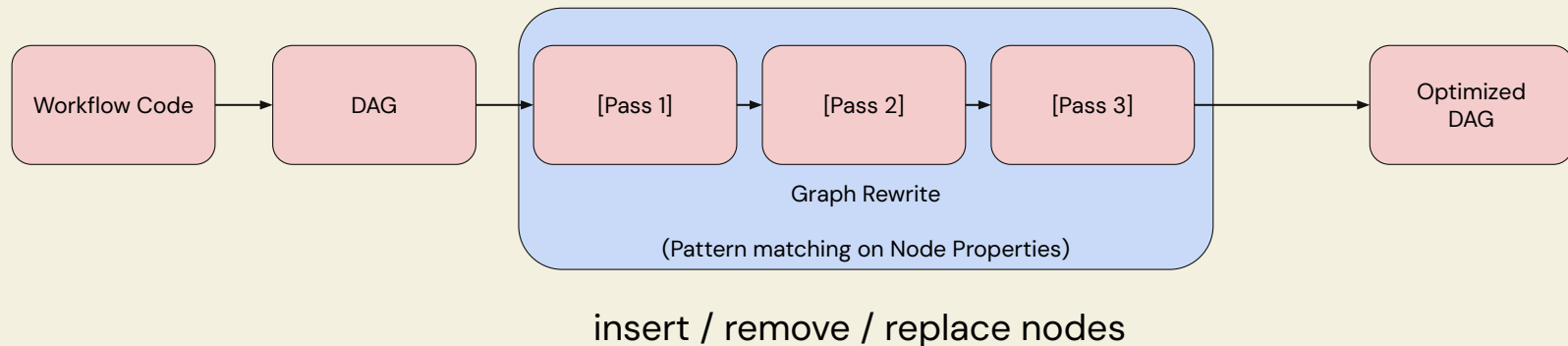
- Independent nodes → run in parallel
- Nodes calling the same model → batch together



Expressed
naturally in
Python



Compiler Optimization Passes (Optional)



Approximate caching: reduces the number of denoising steps by initializing from a pre-cached image of a similar prompt instead of random noise

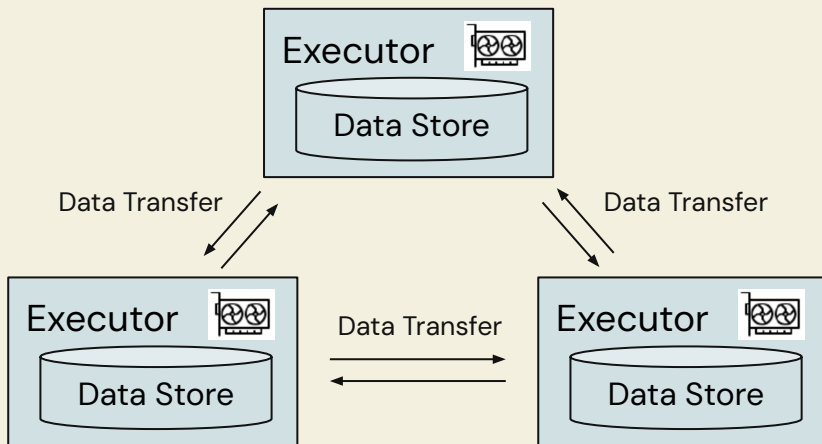
Asynchronous LoRA loading: overlaps LoRA adapter retrieval with the early stages of diffusion model inference.

Data Engine

LegoDiffusion deploys a distributed data engine with a per-executor local data store.

Once the compiler lowers a workflow into nodes, the Data Engine automatically orchestrates all **inter-node data movement**.

Distributed Data Engine



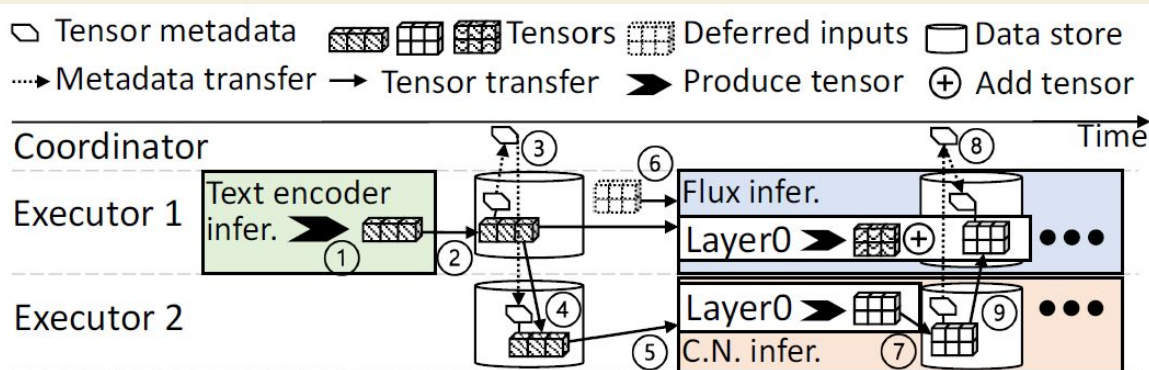
Data stores are built on NVSHMEM

- Zero-copy sharing within an executor
- High-speed transfers across executors
- All intermediate tensors are ***immutable*** → Consistency Protocols & Fault Tolerance

Data Fetch Modes

Eager Mode: Inputs are fetched eagerly: a node cannot begin until all its eager inputs are available.

Deferred Mode: Inputs need not be ready when a node starts inference.



Eager fetch: Executer 2 fetches *prompt embedding* (grid icon) from Executer 1

Deferred fetch: Executer 1 fetches *ControlNet output* (dashed grid icon) from Executer 2

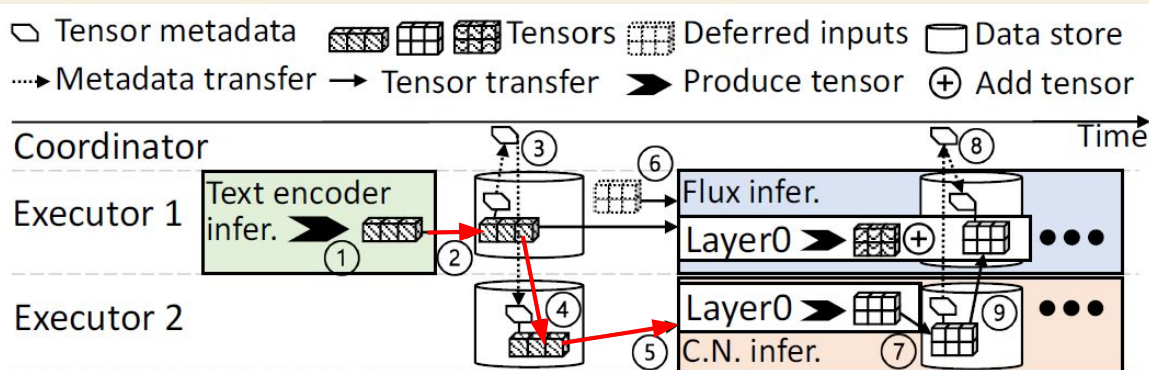
Flux Infer.: Flux main diffusion backbone inference

C.N. Infer.: Control Net Inference

Data Fetch Modes

Eager Mode: Inputs are fetched eagerly: a node cannot begin execution until all its eager inputs are available.

Example: prompt embedding to Control net must be available before Control Net node begins execution



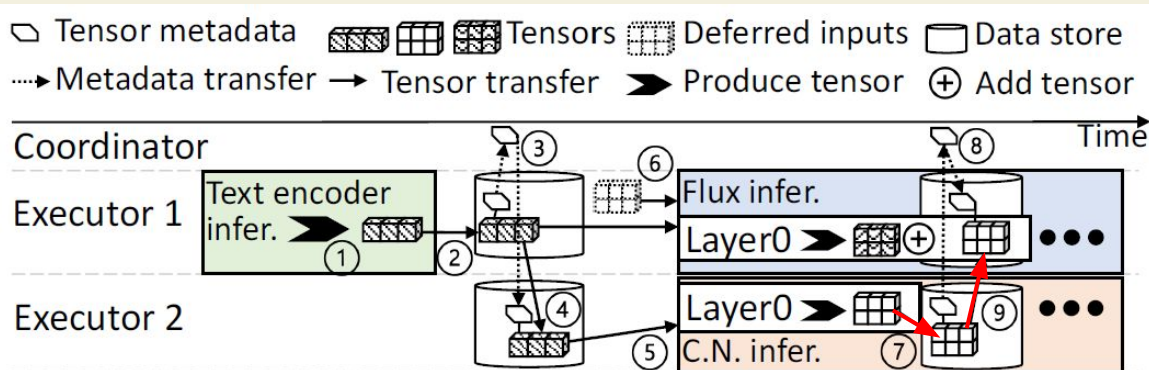
Flux Infer.: Flux main diffusion backbone inference

C.N. Infer.: Control Net Inference

Data Fetch Modes

Deferred Mode: Inputs need not be ready when a node starts inference.

Example: Flux node can begin execution before Control Net output is ready, allowing **inter-node parallelism**.



Eager fetch: Executer 2 fetches *prompt embedding* (tensor) from Executer 1

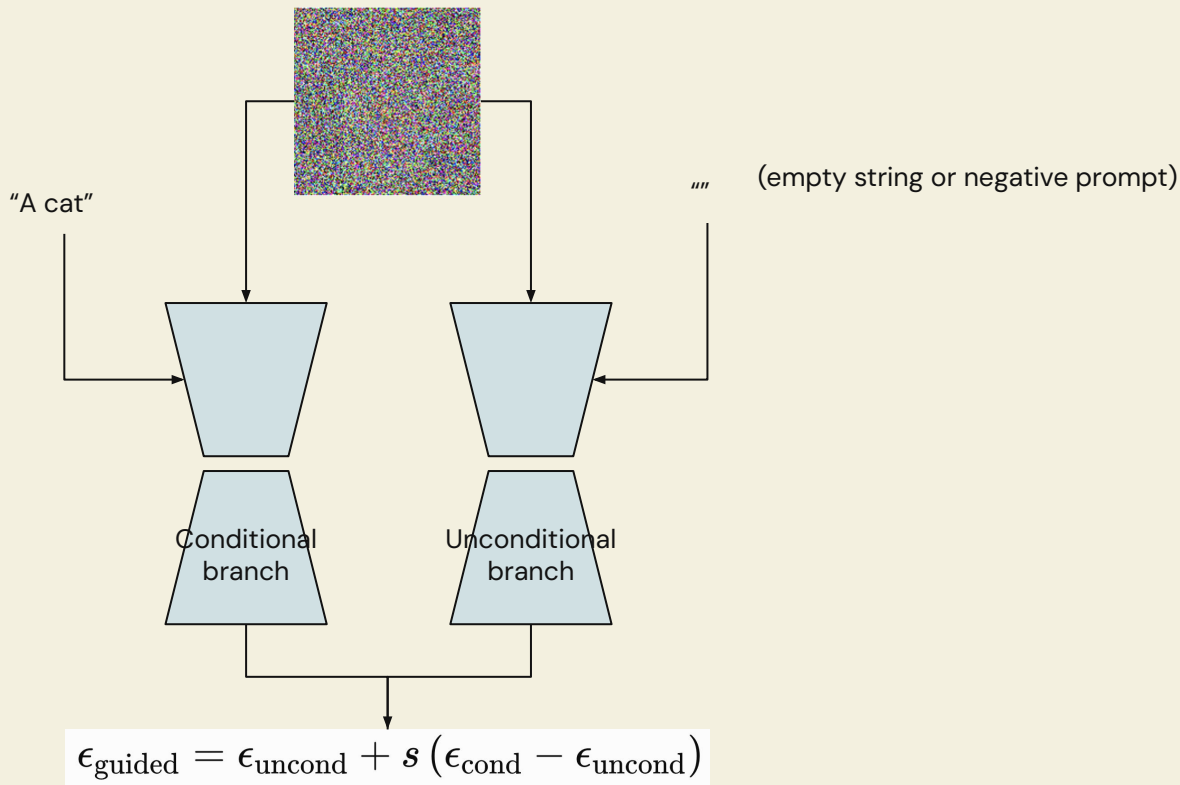
Deferred fetch: Executer 1 fetches *ControlNet output* (deferred input) from Executer 2

Flux Infer.: Flux main diffusion backbone inference

C.N. Infer.: Control Net Inference

Background on Intra-node parallelism

Classifier-free Guidance



Scheduler

Algorithm 1: Scheduling Algorithm

```
1 while True do
    // Admission control runs asynchronously (§5.3)
2   Admit or reject arrived requests;
    // Identify ready nodes
3    $Q_{ready} \leftarrow$  nodes with satisfied dependencies from queue;
4    $E_{avail} \leftarrow$  currently available executors;
5   if  $Q_{ready} = \emptyset$  or  $E_{avail} = \emptyset$  then
6     continue;
7   Sort  $Q_{ready}$  by (arrival time, node depth);
8    $n_{head} \leftarrow Q_{ready}.pop(0)$ ;
    // Batch same-model nodes (§5.1)
9    $B_{max} \leftarrow$  profiled max batch size for  $n_{head}.model$ ;
10   $Batch \leftarrow \{n_{head}\} \cup \{n' \in Q_{ready} \mid n'.model =$ 
     $n_{head}.model \text{ and } |Batch| < B_{max}\}$ ;
    // Choose parallelism degree (§5.2)
11   $k_{max} \leftarrow$  max useful parallelism for  $n_{head}.model$ ;
12   $k \leftarrow \min(|E_{avail}|, k_{max})$ ;
    // Score and select executors
13  for  $e \in E_{avail}$  do
14     $L_{data} \leftarrow \text{CalcDataFetchLatency}(Batch, e)$ ;
15     $L_{load} \leftarrow e \text{ hosts } n_{head}.model ? 0 :$ 
     $\text{CalcLoadTime}(n_{head}.model)$ ;
16     $L_{infer} \leftarrow \text{CalcInferenceTime}(Batch, e, k)$ ;
17     $e.score \leftarrow L_{data} + L_{load} + L_{infer}$ ;
18   $E_{target} \leftarrow$  top  $k$  from  $E_{avail}$  with min scores;
19  Dispatch  $Batch$  to  $E_{target}$  (triggers model load if needed);
```

- The scheduler orders the ready queue by first-come-first-serve (FCFS)
- **Model sharing**: scans the remaining ready nodes for any that reference the same model, and groups them into a single batch of up to B_{max} entries.
- Determine **intra-node parallelism**. $K_{max} = 2$ for classifier-free guidance.
- Score and select k executors with lowest scores and dispatch to them.

Score = cost of fetching data + cost of loading model + estimated inference time

Setup & Evaluation

1. Does micro-serving beat monolithic serving?
2. How much does each strategy (Model sharing, adaptive parallelism) contribute?
3. How much overhead does decomposition introduce?

8-32
H800 GPUs

FastAPI frontend
(1000 LoC Python)

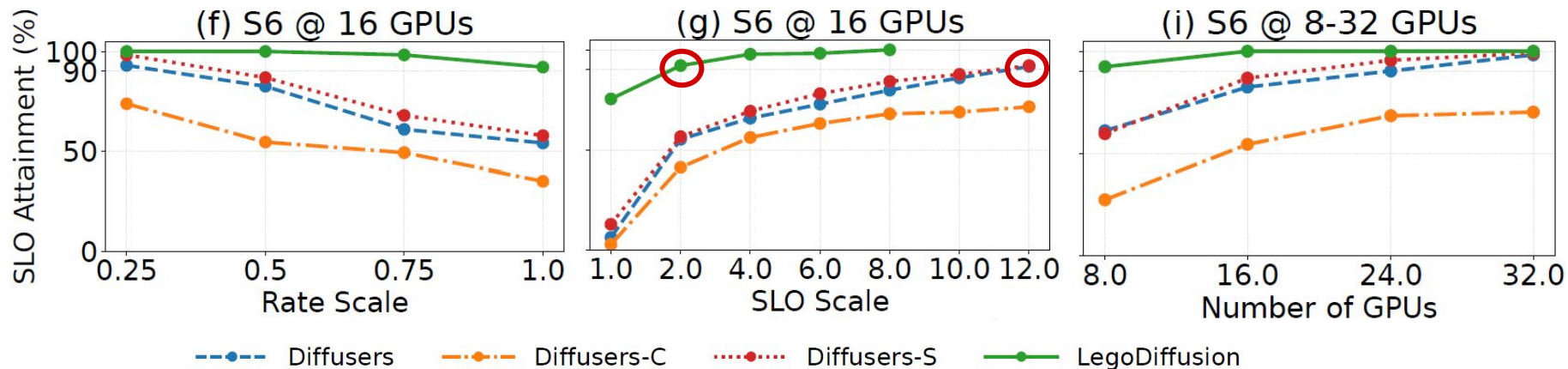
C++/CUDA data engine
(NVSHMEM)

Setting	Base model(s)	Workflows
S1	SD3	Basic, +1 ControlNet, +2 ControlNets
S2	SD3.5-Large	same 3 variants
S3	Flux-Schnell	same 3 variants
S4	Flux-Dev	same 3 variants
S5	SD3 + SD3.5-Large	6 (S1 + S2)
S6	Flux-Schnell + Flux-Dev	6 (S3 + S4)

Baseline Framework: Huggingface Diffusers, Diffusers-C, Diffusers-S (**all Monolithic**)

Results

SLO attainment: the fraction of requests completed within their specified latency deadline



3x Higher Request Rate

At 90% SLO attainment

6x more stringent SLOs

satisfied

3x fewer GPUs

Microbenchmarks

Model Sharing

4.5x less memory & faster
latency

Intra-node parallelism

1.9x Inference Speedup

Inter-node parallelism

1.3x Inference Speedup

Execution Overhead

150 ms max
(total workload $\approx$ 2-20s)

Control-plane Scalability

$\sim$ 3% of total execution time

Data Transmission Latency

< 1 ms

Takeaways & discussion

Monolithic T2I workflows → coarse scaling, no sharing, rigid parallelism

Solution: Micro-serving

1 Make the model the unit of scheduling

Per-model scaling, sharing and parallelism all follow from one decomposition

2 The data plane ensures improvement

GPU-native transfers and deferred fetch minimizes overhead and preserve ControlNet parallelism.

3 Objective: on-time completions

Up to 3× request rate, but gains depend on model reuse and the baseline.

Practical limits

- High model-reuse workflows
- Large scale 256-GPU results are simulated

Questions

- How much is the overhead for few step models?
- How much model reuse does a real deployment have?

Thank you